



THAPAR INSTITUTE  
OF ENGINEERING & TECHNOLOGY  
(Deemed to be University)

## ASSIGNMENT 7

Aadhya Maheshwari—1024150212

### 1. Swap two variables of any data type

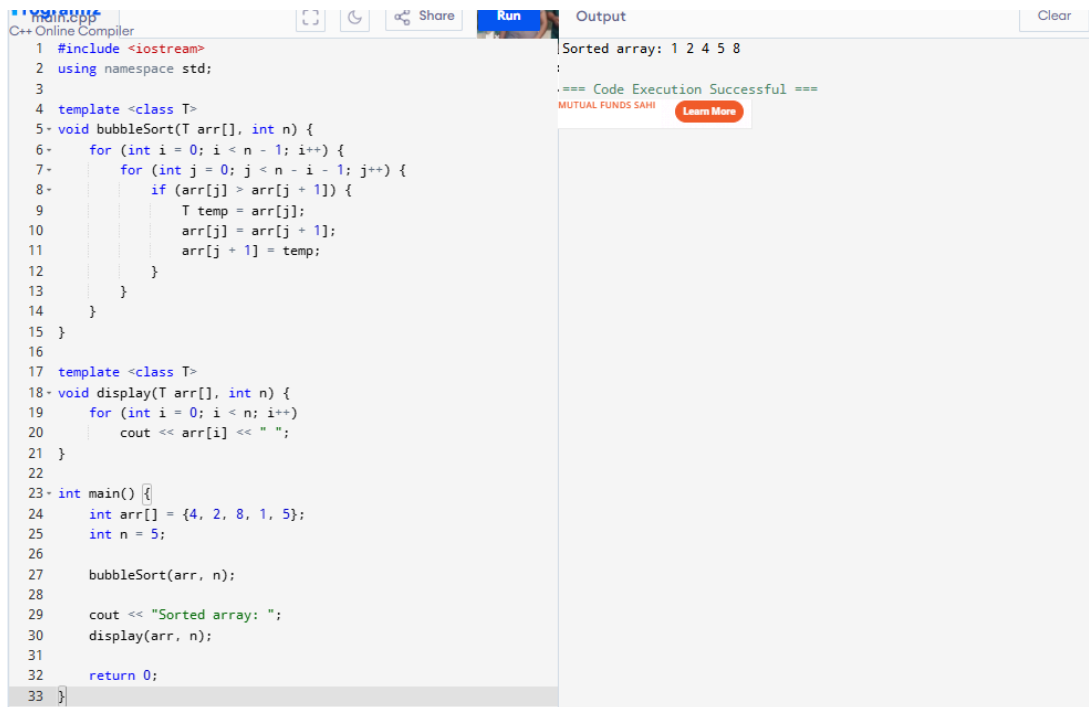
| main.cpp                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Output                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <pre>1 #include &lt;iostream&gt; 2 using namespace std; 3 4 template &lt;class T&gt; 5 void swapValues(T &amp;a, T &amp;b) { 6     T temp = a; 7     a = b; 8     b = temp; 9 } 10 11 int main() { 12     int x = 10, y = 20; 13 14     cout &lt;&lt; "Before swap: " &lt;&lt; x &lt;&lt; " " &lt;&lt; y &lt;&lt; endl; 15     swapValues(x, y); 16     cout &lt;&lt; "After swap: " &lt;&lt; x &lt;&lt; " " &lt;&lt; y &lt;&lt; endl; 17 18     return 0; 19 }</pre> | <pre>Before swap: 10 20 After swap: 20 10  === Code Execution Successful ===</pre> |

### 2. Find minimum element in an array

| main.cpp                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Output                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <pre>1 #include &lt;iostream&gt; 2 using namespace std; 3 4 template &lt;class T&gt; 5 T findMin(T arr[], int n) { 6     T minVal = arr[0]; 7 8     for (int i = 1; i &lt; n; i++) { 9         if (arr[i] &lt; minVal) 10             minVal = arr[i]; 11     } 12 13     return minVal; 14 } 15 16 int main() { 17     int arr[] = {5, 2, 9, 1, 7}; 18     int n = 5; 19 20     cout &lt;&lt; "Minimum element: " &lt;&lt; findMin(arr, n); 21 22     return 0; 23 }</pre> | <pre>Minimum element: 1  === Code Execution Successful ===</pre> |

---

### 3. Sort an array using bubble sort



The screenshot shows a C++ online compiler interface. The code implements a bubble sort algorithm. It includes `<iostream>` and uses the `std` namespace. A template function `bubbleSort` is defined, which takes an array `arr` and its size `n`. It uses nested loops: the outer loop iterates from `i = 0` to `n - 1`, and the inner loop iterates from `j = 0` to `n - i - 1`. Inside the inner loop, it compares `arr[j]` and `arr[j + 1]`. If `arr[j] > arr[j + 1]`, it swaps them using a temporary variable `temp`. A `display` function is also defined, which prints the array elements. In the `main` function, an array `arr` is initialized with `{4, 2, 8, 1, 5}` and `n` is set to 5. `bubbleSort(arr, n)` is called, and the sorted array is displayed. The output shows the sorted array: 1 2 4 5 8.

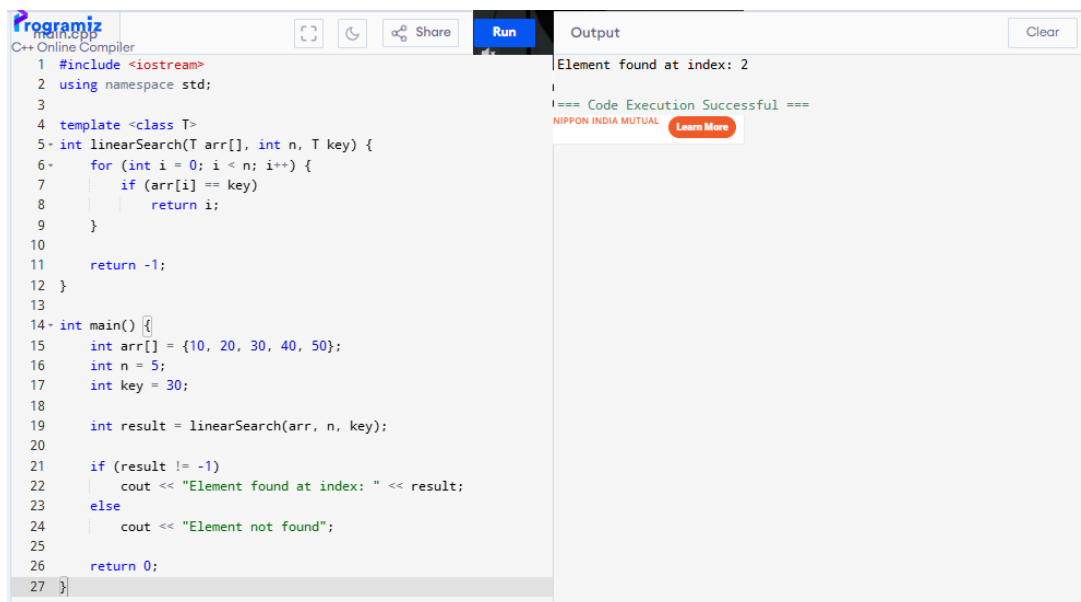
```
1 #include <iostream>
2 using namespace std;
3
4 template <class T>
5 void bubbleSort(T arr[], int n) {
6     for (int i = 0; i < n - 1; i++) {
7         for (int j = 0; j < n - i - 1; j++) {
8             if (arr[j] > arr[j + 1]) {
9                 T temp = arr[j];
10                arr[j] = arr[j + 1];
11                arr[j + 1] = temp;
12            }
13        }
14    }
15 }
16
17 template <class T>
18 void display(T arr[], int n) {
19     for (int i = 0; i < n; i++)
20         cout << arr[i] << " ";
21 }
22
23 int main() {
24     int arr[] = {4, 2, 8, 1, 5};
25     int n = 5;
26
27     bubbleSort(arr, n);
28
29     cout << "Sorted array: ";
30     display(arr, n);
31
32     return 0;
33 }
```

Sorted array: 1 2 4 5 8

Code Execution Successful

---

### 4. Perform linear search



The screenshot shows a C++ online compiler interface. The code implements a linear search algorithm. It includes `<iostream>` and uses the `std` namespace. A function `linearSearch` is defined, which takes an array `arr`, its size `n`, and a `key`. It iterates from `i = 0` to `n - 1`. If `arr[i] == key`, it returns `i`. If the key is not found, it returns `-1`. In the `main` function, an array `arr` is initialized with `{10, 20, 30, 40, 50}`, `n` is set to 5, and `key` is set to 30. `linearSearch(arr, n, key)` is called, and the result is stored in `result`. If `result` is not `-1`, it prints "Element found at index: " followed by `result`. Otherwise, it prints "Element not found". The output shows "Element found at index: 2".

```
1 #include <iostream>
2 using namespace std;
3
4 template <class T>
5 int linearSearch(T arr[], int n, T key) {
6     for (int i = 0; i < n; i++) {
7         if (arr[i] == key)
8             return i;
9     }
10
11     return -1;
12 }
13
14 int main() {
15     int arr[] = {10, 20, 30, 40, 50};
16     int n = 5;
17     int key = 30;
18
19     int result = linearSearch(arr, n, key);
20
21     if (result != -1)
22         cout << "Element found at index: " << result;
23     else
24         cout << "Element not found";
25
26     return 0;
27 }
```

Element found at index: 2

Code Execution Successful

---

## 5. Overloaded function template process()

| main.cpp                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Output                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>1 #include &lt;iostream&gt; 2 using namespace std; 3 4 template &lt;class T&gt; 5 void process(T a) { 6     cout &lt;&lt; "Single parameter: " &lt;&lt; a &lt;&lt; endl; 7 } 8 9 template &lt;class T&gt; 10 void process(T a, T b) { 11     cout &lt;&lt; "Two parameters of same type: " &lt;&lt; a &lt;&lt; " " &lt;&lt; b &lt;&lt;         endl; 12 } 13 14 template &lt;class T, class U&gt; 15 void process(T a, U b) { 16     cout &lt;&lt; "Two parameters of different types: " &lt;&lt; a &lt;&lt; " " &lt;&lt;         b &lt;&lt; endl; 17 } 18 19 int main() { 20     process(10); 21     process(10, 20); 22     process(10, 5.5); 23 24     return 0; 25 }</pre> | <pre>Single parameter: 10 Two parameters of same type: 10 20 Two parameters of different types: 10 5.5  === Code Execution Successful ===</pre> |

---

# Class Templates

## 1. Stack with push and pop operations

main.cpp

Share

Run

```
1 #include <iostream>
2 using namespace std;
3
4 template <class T>
5 class Stack {
6     T arr[100];
7     int top;
8
9 public:
10     Stack() {
11         top = -1;
12     }
13
14     void push(T value) {
15         if (top == 99)
16             cout << "Stack Overflow" << endl;
17         else
18             arr[++top] = value;
19     }
20
21     void pop() {
22         if (top == -1)
23             cout << "Stack Underflow" << endl;
24         else
25             cout << "Popped element: " << arr[top--] << endl;
26     }
27
28     void display() {
29         if (top == -1)
30             cout << "Stack is empty" << endl;
31         else {
32             for (int i = top; i >= 0; i--)
33                 cout << arr[i] << " ";
34             cout << endl;
35         }
36     }
37 };
38
```

Output

```
30 20 10
Popped element: 30
20 10

=== Code Execution Successful ===
```

```
39 int main() {
40     Stack<int> s;
41
42     s.push(10);
43     s.push(20);
44     s.push(30);
45
46     s.display();
47     s.pop();
48     s.display();
49
50     return 0;
51 }
```

## 2. Queue with enqueue and dequeue operations

```
1 #include <iostream>
2 using namespace std;
3
4 template <class T>
5 class Queue {
6     T arr[100];
7     int front, rear;
8
9 public:
10    Queue() {
11        front = -1;
12        rear = -1;
13    }
14
15    void enqueue(T value) {
16        if (rear == 99)
17            cout << "Queue Overflow" << endl;
18        else {
19            if (front == -1)
20                front = 0;
21            arr[++rear] = value;
22        }
23    }
24
25    void dequeue() {
26        if (front == -1 || front > rear)
27            cout << "Queue Underflow" << endl;
28        else
29            cout << "Dequeued element: " << arr[front++] <<
                endl;
30    }
31
32    void display() {
33        if (front == -1 || front > rear)
34            cout << "Queue is empty" << endl;
35        else {
36            for (int i = front; i <= rear; i++)
37                cout << arr[i] << " ";
38            cout << endl;
39        }
40    }
41 };
42
43 int main() {
44     Queue<int> q;
45
46     q.enqueue(10);
47     q.enqueue(20);
48     q.enqueue(30);
49
50     q.display();
51     q.dequeue();
52     q.display();
53
54     return 0;
55 }
```

10 20 30  
:Dequeued element: 10  
20 30

=== Code Execution Successful ===

### 3. Store and display a pair of values

```
1 #include <iostream>
2 using namespace std;
3
4 template <class T, class U>
5 class Pair {
6     T first;
7     U second;
8
9 public:
10     Pair(T a, U b) {
11         first = a;
12         second = b;
13     }
14
15     void display() {
16         cout << "First value: " << first << endl;
17         cout << "Second value: " << second << endl;
18     }
19 };
20
21 int main() {
22     Pair<int, double> p(10, 5.5);
23
24     p.display();
25
26     return 0;
27 }
```

First value: 10  
Second value: 5.5

=== Code Execution Successful ===

### 4. Basic arithmetic operations

```
1 #include <iostream>
2 using namespace std;
3
4 template <class T>
5 class Arithmetic {
6     T a, b;
7
8 public:
9     Arithmetic(T x, T y) {
10         a = x;
11         b = y;
12     }
13
14     void display() {
15         cout << "Addition: " << a + b << endl;
16         cout << "Subtraction: " << a - b << endl;
17         cout << "Multiplication: " << a * b << endl;
18
19         if (b != 0)
20             cout << "Division: " << a / b << endl;
21         else
22             cout << "Division not possible" << endl;
23     }
24 };
25
26 int main() {
27     Arithmetic<double> obj(10.5, 2.5);
28
29     obj.display();
30
31     return 0;
32 }
```

First value: 10  
Second value: 5.5

=== Code Execution Successful ===

## 5. Input and display array elements

| main.cpp                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Output                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <pre>1 #include &lt;iostream&gt; 2 using namespace std; 3 4 template &lt;class T&gt; 5 class Array { 6     T arr[100]; 7     int n; 8 9 public: 10     void input() { 11         cout &lt;&lt; "Enter number of elements: "; 12         cin &gt;&gt; n; 13 14         cout &lt;&lt; "Enter elements: "; 15         for (int i = 0; i &lt; n; i++) 16             cin &gt;&gt; arr[i]; 17     } 18 19     void display() { 20         cout &lt;&lt; "Array elements are: "; 21         for (int i = 0; i &lt; n; i++) 22             cout &lt;&lt; arr[i] &lt;&lt; " "; 23     } 24 }; 25 26 int main() { 27     Array&lt;int&gt; a; 28 29     a.input(); 30     a.display(); 31 32     return 0; 33 }</pre> | <pre>Enter number of elements: 5 Enter elements: 1 2 3 4 5 Array elements are: 1 2 3 4 5  === Code Execution Successful ===</pre> |